

Database *Design* Principles

Best practices for database design · MySQL · Schema Architecture

Objective: Understand best practices for database design, master fact & dimension tables, Star and Snowflake schemas, entity-attribute modelling, and create tables in MySQL.

SECTION 1 · CORE CONCEPTS

■ Fact Table

The central table in a data warehouse storing quantitative, measurable data (metrics & events). References dimension tables via foreign keys — high volume, numeric.

[Numeric Measures] [Foreign Keys] [High Volume]

Column / Entity	Type / Attributes	Role / Example
sale_id	INT (PK)	Primary key
date_id	INT (FK)	→ dim_date
product_id	INT (FK)	→ dim_product
customer_id	INT (FK)	→ dim_customer
quantity	INT	Measure
total_amount	DECIMAL(10,2)	Measure

■ Dimension Table

Descriptive tables providing context to facts. They answer who, what, where, when. Typically smaller and denormalized — one row per unique entity.

[\[Descriptive\]](#) [\[Denormalized\]](#) [\[Reference Data\]](#)

Column / Entity	Type / Attributes	Role / Example
dim_customer	cust_id, name, city, segment	Who bought
dim_product	prod_id, category, brand, price	What was sold
dim_date	date_id, day, month, quarter, year	When it happened
dim_store	store_id, location, region, size	Where it happened

■■ Entity & Attributes

An entity is a real-world object stored as a database table (e.g. Student, Order). Attributes are its properties — each becomes a column with a name and data type.

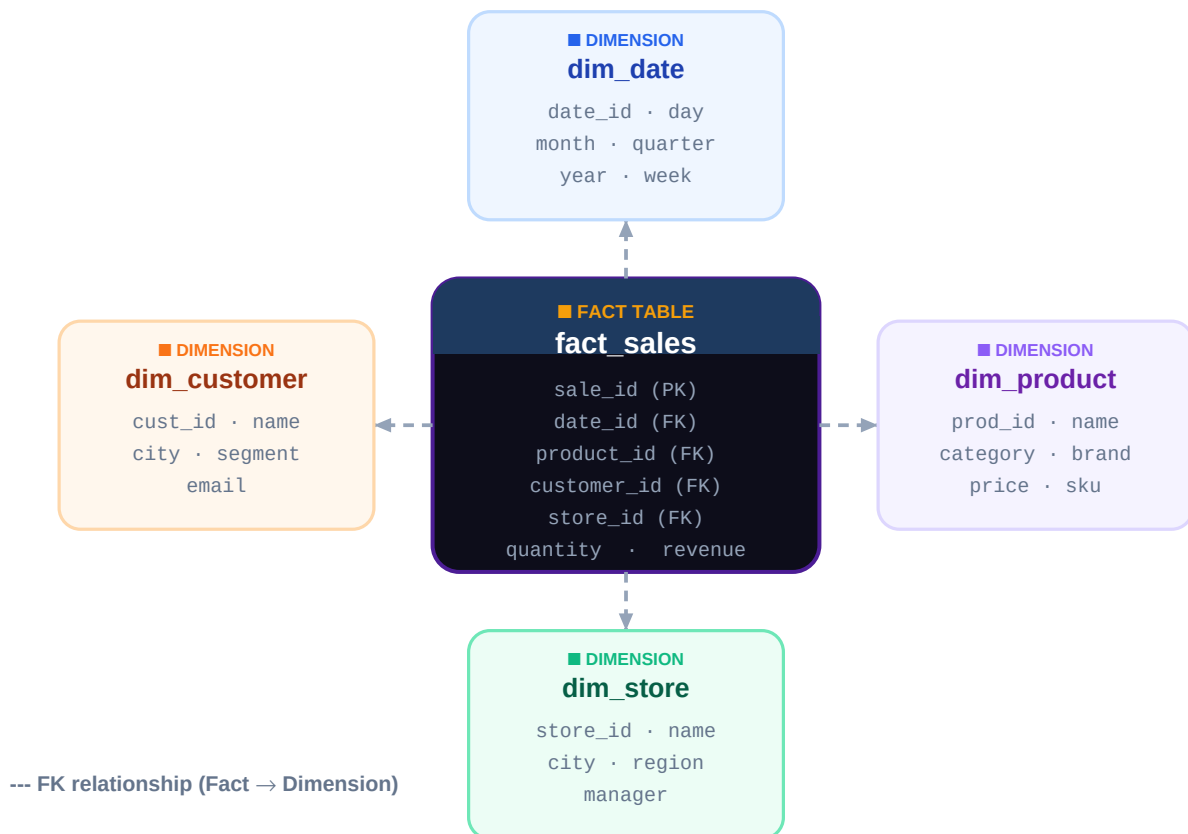
[\[Entity = Table\]](#) [\[Attribute = Column\]](#) [\[PK per Entity\]](#)

Column / Entity	Type / Attributes	Role / Example
Student	student_id (PK), name, email, dob	Academic record
Course	course_id (PK), title, credits, dept	Curriculum
Order	order_id (PK), customer_id (FK), date	Transaction
Employee	emp_id (PK), dept_id (FK), salary, hiredate	HR data

Star Schema

Denormalized

One central **Fact Table** connected directly to all **Dimension Tables** — like a star shape. Dimensions are flat (not normalized further). Optimized for **fast query performance** in BI and reporting workloads.



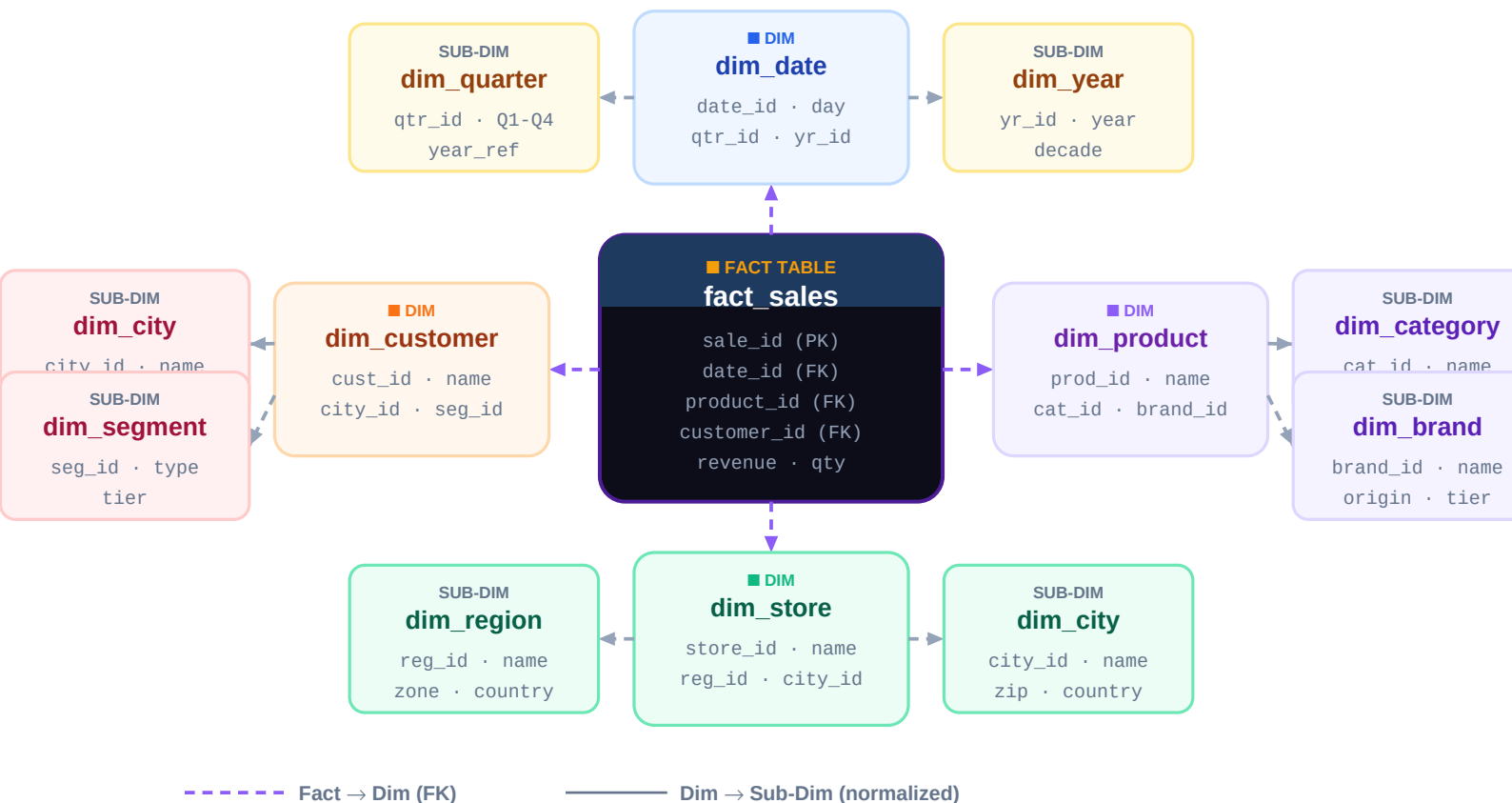
- ✓ Fewer JOINS — faster queries
- ✓ Simple to understand & query
- ✓ Ideal for BI & reporting tools
- ✓ Works great with OLAP cubes

- ✗ Data redundancy in dimensions
- ✗ Higher storage usage
- ✗ Harder to update repeated values
- ✗ Denormalized = not 3NF

Structure	Query Pattern	Best For	Normal Form
1 Fact + N flat Dims	Simple JOINS · 2 hops max	OLAP · BI · Dashboards	Denormalized

❄️ Snowflake Schema

An extension of the Star Schema where **dimension tables are further normalized** into sub-dimension tables — branching out like a snowflake crystal. Eliminates redundancy at the cost of more complex JOIN queries.



- ✓ Eliminates data redundancy
- ✓ Saves storage space
- ✓ Easier to update dimension values
- ✓ Conforms to 3NF

- ✗ More complex JOIN queries
- ✗ Slower query performance
- ✗ Harder to understand schema
- ✗ More tables to manage

Structure	Query Pattern	Best For	Normal Form
Fact + Dims + Sub-Dims	Multi JOINS · 3+ hops	Large DWH · Storage-sensitive	Normalized (3NF)

Criteria	■ Star Schema	❄️ Snowflake Schema
Normalization	Denormalized	Fully Normalized (3NF)
Query Performance	Faster	Slower
Storage Usage	Higher	Lower
Data Redundancy	Present	Eliminated
Number of Tables	Fewer	More (sub-dims added)
JOIN Complexity	Simple (2 hops)	Complex (3+ hops)
Ease of Understanding	Easy	Complex
Typical Use Case	BI, Dashboards, OLAP	Large DWH, Data Marts
ETL Complexity	Simpler	More complex
Maintenance	Easier	Harder (more tables)

When to use Star Schema: Your team runs heavy BI/dashboard queries and query speed is the priority. Storage cost is acceptable and your dimension data doesn't change often.

When to use Snowflake Schema: You have large dimension tables with repeating values, storage is a concern, or you need strict data integrity and normalization for compliance.

■ Understand Business Requirements

Identify entities, attributes, and relationships clearly. Know what data to store and why before writing a single line of SQL.

#01

■ Identify Proper Entities (Tables)

One table = one real-world object (Student, Course, Order). Avoid mixing multiple concepts in one table.

#02

■ Choose Correct Datatypes

INT for IDs, VARCHAR for text, DATE/TIMESTAMP for dates. Avoid TEXT when VARCHAR is sufficient.

#03

■ Define Primary Keys

Every table must have a PRIMARY KEY. Use surrogate keys (id) where natural keys are complex or composite.

#04

■ Normalize the Database

Remove redundant data. Follow 1NF (atomic), 2NF (no partial deps), 3NF (no transitive deps). Split repeating columns.

#05

■ Plan Relationships

Identify One-to-One, One-to-Many, Many-to-Many. Use FOREIGN KEYS to enforce referential integrity.

#06

■ Avoid Redundancy

Do not duplicate columns across tables unnecessarily. Store data once and reference it using foreign keys.

#07

→ ■ Meaningful Naming Conventions

Tables: plural nouns (Students, Orders). Columns: snake_case, self-explanatory. Avoid SQL reserved keywords.

#08

■ Handle NULL Values Carefully

Allow NULL only when data is truly optional. Use NOT NULL where data is mandatory for business logic.

#09

■ Scalability & Performance

Index columns used in JOIN, WHERE, GROUP BY. Avoid over-indexing as it slows INSERT/UPDATE operations.

#10

■ Security & Data Sensitivity

Separate sensitive data (passwords, PII) into protected tables. Use proper access control and encryption.

#11

■ Plan for Future Changes

Design a flexible schema. Avoid storing hard-coded or derived data. Schema should adapt to business evolution.

#12

MySQL Data Types

Use Case	MySQL Type	Example Column
Identifier / Counter	INT	student_id, order_id
Short text	VARCHAR(n)	name, email, title
Long free text	TEXT	description, notes
Exact decimal	DECIMAL(p,s)	price, salary, total
Date only	DATE	birth_date, hire_date
Date + Time	TIMESTAMP	created_at, updated_at
Boolean flag	TINYINT(1)	is_active, is_deleted
Fixed choices	ENUM(...)	status, gender, role
Binary / file data	BLOB	profile_photo

Relationship Types & Normal Forms

Type	Example	Implementation	Key Point
1 : 1	User ↔ Profile	FK + UNIQUE constraint	Each side has at most one match
1 : M	Dept → Employees	FK on the 'many' table	Most common relationship
M : M	Students ↔ Courses	Junction / bridge table	Junction holds both FKs + extras
Self-ref	Employee → Manager	FK references same table	Hierarchical / tree structures

Normal Form	Core Rule	Example Fix
1NF — First	Atomic values; no repeating groups or arrays	Separate phone1, phone2 into a Phone:
2NF — Second	No partial dependency on a composite primary key	Move course_name out of Enrollment i
3NF — Third	No transitive dependency (non-key → non-key)	Move dept_location out of Employees i

Lecture 01 · Database Design Principles & Creating Tables in MySQL · Database Systems Course · All principles apply to relational database design